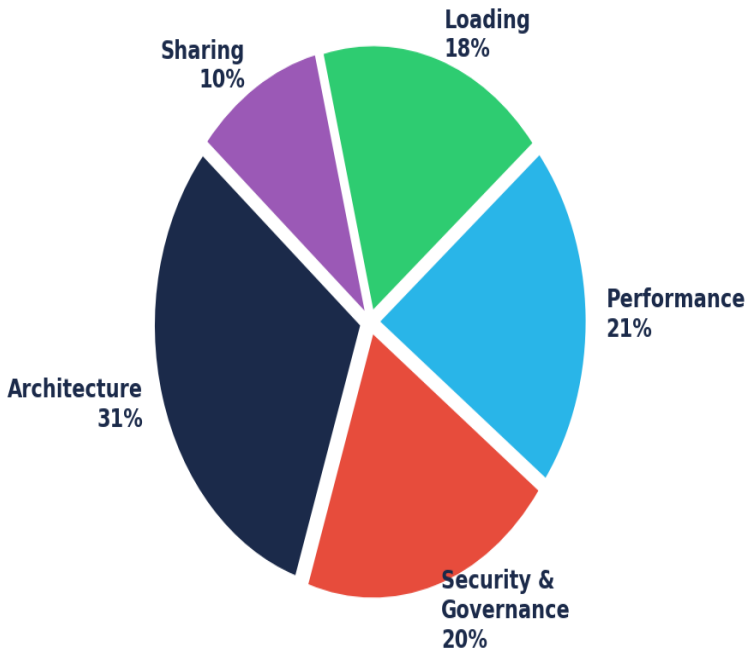
SNOWPRO CORE (COF-C03)

MASTER CHEAT SHEET

100 Questions | 115 Minutes | Pass: 750/1000 | $175 USD

COF-C03 EXAM DOMAIN WEIGHTS



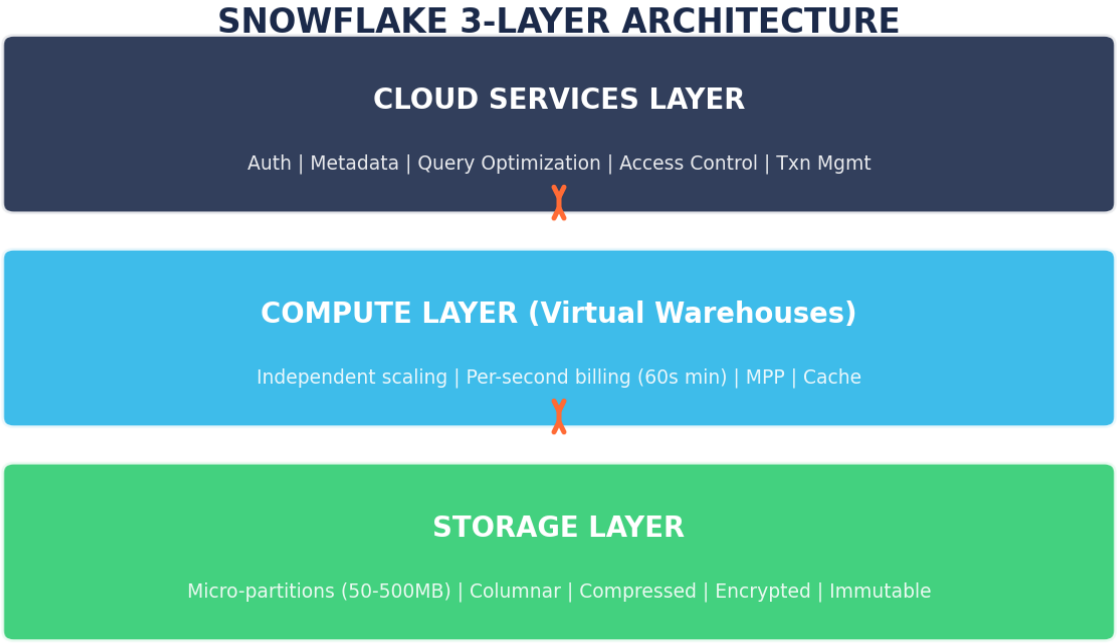
ALL EDITIONS: Snowpipe, Streams, Tasks, Dynamic Tables, SCIM, MFA, Snowpark, SSO, OAuth, Key Rotation, Clustering Keys

ENTERPRISE+: Multi-cluster WH, Mat Views, Search Opt, Query Accel, Column/Row Security, 90-day TT, Periodic Rekeying, Data Classification

BUSINESS CRITICAL+: PrivateLink, Tri-Secret Secure, HIPAA/PCI/SOC, Failover/Failback, ISMAP

https://learn.snowflake.com/en/certifications/snowpro-core-c03/

1. ARCHITECTURE & AI DATA CLOUD FEATURES (31%)



- **Hybrid Architecture**: shared-disk (central data) + shared-nothing (MPP compute). Combines data mgmt simplicity with scale-out performance.

https://docs.snowflake.com/en/user-guide/intro-key-concepts#snowflake-architecture

Cloud Services Layer Details

- Security, authentication, access control
- Snowflake Horizon Catalog (metadata/governance)
- Query parsing and optimization, transaction management
- Infrastructure management with cloud platforms
- Billed only if >10% of total daily warehouse compute credits

https://docs.snowflake.com/en/user-guide/intro-key-concepts#cloud-services

Micro-Partitions & Data Clustering

- 50-500 MB **uncompressed** (smaller after compression), columnar, immutable, auto-created
- Pruning = skip unneeded partitions (check: partitions scanned vs total in Query Profile)
- Natural clustering = order data was inserted; explicit clustering = CLUSTER BY
- SYSTEM$CLUSTERING_DEPTH() and SYSTEM$CLUSTERING_INFORMATION() to evaluate
- Clustering keys: CANNOT use GEOGRAPHY, VARIANT, OBJECT, ARRAY types; medium cardinality best; depth=1 optimal

https://docs.snowflake.com/en/user-guide/tables-clustering-micropartitions

MPP (Massively Parallel Processing)

- Each virtual warehouse = independent MPP compute cluster
- Nodes don't share resources with other warehouses
- Warehouses have no impact on each other's performance
- Each node stores a portion of the data set locally (caching)

https://docs.snowflake.com/en/user-guide/intro-key-concepts#compute

Edition Feature Matrix

EDITION FEATURE MATRIX

	Standard	Enterprise	Business Critical	VPS
Time Travel (90d)	X	Y	Y	Y
Multi-cluster WH	X	Y	Y	Y
Materialized Views	X	Y	Y	Y
Search Optimization	X	Y	Y	Y
Column/Row Security	X	Y	Y	Y
Periodic Rekeying	X	Y	Y	Y
Tri-Secret Secure	X	X	Y	Y
PrivateLink	X	X	Y	Y
HIPAA/PCI	X	X	Y	Y
Failover/Failback	X	X	Y	Y
Dedicated Resources	X	X	X	Y

https://docs.snowflake.com/en/user-guide/intro-editions

Key Numbers -- MEMORIZE!

What	Value
Warehouse min billing per start	60 seconds (then per-second)
Result cache expiry	24 hours (max 31 days with reuse)
Metadata cache	Always on, Cloud Services, row count/min/max/etc.
ACCOUNT_USAGE latency / retention	45min-3hr / 365 days
INFORMATION_SCHEMA retention	7-14 days (real-time, no latency)
COPY INTO load history tracking	64 days
Fail-safe duration	7 days (not configurable, permanent tables only)
Time Travel max (Enterprise+)	90 days (Standard = 0-1)
VARIANT row limit	128 MB uncompressed
Snowpipe recommended file size	100-250 MB compressed
Stream stale default	14 days (= DATA_RETENTION_TIME_IN_DAYS + offset)
Task timeout default	60 minutes
Session idle timeout default	240 min (configurable, min: 5 min)
Max columns per table	10,000
Max clustering keys	4 columns recommended

Object Hierarchy

Account-Level	Database-Level	Schema-Level
Warehouses	Schemas	Tables, Views, Mat Views
Resource Monitors	Database Roles	Pipes, File Formats
Storage Integrations		Stages, Sequences
API Integrations		UDFs, Stored Procedures
Users, Roles		Streams, Tasks
Databases		Dynamic Tables, ML Models
Network Policies		Masking/Row Access Policies
Shares, Replication Groups		Tags

https://docs.snowflake.com/en/user-guide/intro-key-concepts

Session & Context Variables, Parameter Hierarchy

Concept	Detail
CURRENT_ROLE()	Active primary role for the session
CURRENT_WAREHOUSE()	Active warehouse (can be NULL)
CURRENT_DATABASE() / CURRENT_SCHEMA()	Active DB/schema context
CURRENT_SESSION()	Unique session ID
CURRENT_USER()	Logged-in user
CURRENT_ACCOUNT()	Account locator (short form)
CURRENT_REGION()	Cloud region
CURRENT_ORGANIZATION_NAME()	Organization name
CURRENT_ACCOUNT_NAME()	Account name (within org)
Parameter hierarchy	Account > User > Session > Object (most specific wins)
SET / UNSET	Session variables: SET myvar = 10; SELECT $myvar
SHOW PARAMETERS	SHOW PARAMETERS IN ACCOUNT/SESSION/WAREHOUSE x

https://docs.snowflake.com/en/sql-reference/functions/current_role

Virtual Warehouse Types & Configuration

WH Type	Details
Standard (Gen 1 & Gen 2)	Default; Gen1: XS-6XL; Gen2: XS-4XL only
Snowpark-Optimized	16x more memory for ML/Snowpark; min size = Medium (default); X-Small with MEMORY_1X
Sizing: XS to 6XL	Each size UP = 2x credits/sec, ~2x performance
Auto-suspend	Rec 5-10 min; 0 = never suspend; min = 60s
Auto-resume	Default ON; resumes when query hits warehouse
Initially suspended	Default OFF; warehouse starts running at creation

- Resize does NOT affect currently running queries; only new queries use new size
- Suspend = local cache WIPED (warehouse disk cache lost)
- USE WAREHOUSE x sets context; queries without warehouse = error (except metadata-only)

https://docs.snowflake.com/en/user-guide/warehouses-overview

Table Types (Complete)

Type	Persists	Time Travel	Fail-safe	Notes
Permanent	Yes	0-90 days	7 days	Default type, production data
Transient	Yes	0-1 day	NONE	CREATE TRANSIENT TABLE; lower cost
Temporary	Session only	0-1 day	NONE	Dropped at session end; ETL scratch
External	Metadata only	None	None	Data stays in ext storage; read-only
Dynamic	Yes	Same as base	Same as base	Auto-refreshing via TARGET_LAG; no DML
Iceberg	Yes	Yes	NO fail-safe	Open format, Parquet, interoperable
Hybrid	Yes	Yes	Yes	Operational + analytical (preview)

https://docs.snowflake.com/en/user-guide/tables-overview

Apache Iceberg Tables (NEW to COF-C03)

- **External Volume** = account-level object connecting Snowflake to cloud storage (S3, GCS, Azure)
- **Catalog Integration** = account-level object for external catalog metadata (AWS Glue, Open Catalog)
- Two catalog options: (1) Snowflake as catalog = full support, read+write; (2) External catalog = limited
- Data stored in Apache Parquet format; NO Fail-safe; NO temporary/transient Iceberg tables
- Snowflake-managed: supports clustering, replication, standard+append-only streams
- Externally managed: NO clustering, NO standard streams (insert-only only), NO replication

https://docs.snowflake.com/en/user-guide/tables-iceberg

External Tables

- Read-only table referencing data in external stage; metadata in Snowflake, data stays external
- Supports auto-refresh via event notifications
- Can create materialized views on external tables (Enterprise+)
- Data in VALUE column (VARIANT type) + METADATA$FILENAME + METADATA$FILE_ROW_NUMBER

https://docs.snowflake.com/en/user-guide/tables-external-intro

View Types

Type	Key Detail
Standard View	Definition visible; CAN be shared (secure recommended)
Secure View	Hides definition + optimizer; REQUIRED for sharing; slightly slower
Materialized View	Pre-computed, auto-refreshed (serverless); Enterprise+; single table, NO joins, NO UDTFs

- ALTER VIEW cannot change columns -- must CREATE OR REPLACE

https://docs.snowflake.com/en/user-guide/views-introduction

Snowsight & Tools

- Each worksheet = unique session | Folders CANNOT be nested
- Charts: Bar, Line, Scatter, Heat grid, Scorecard (NO pie/box plot in Snowsight)
- Partner Connect: ACCOUNTADMIN required, auto-creates DB/WH/user/role
- **SnowSQL**=legacy CLI (PUT/GET) | **SnowCD**=connectivity diagnostics | **SQL API**=REST+JSON+OAuth
- **Snowflake CLI (snow)**=modern CLI for deploy, streamlit, notebooks, git
- Default inbound shares: SNOWFLAKE (ACCOUNT_USAGE) + SNOWFLAKE_SAMPLE_DATA
- PUT command CANNOT be run from Snowsight -- use SnowSQL or drivers
- **Drivers** = low-level (JDBC, ODBC) | **Connectors** = higher-level (Kafka, Spark, Python)

https://docs.snowflake.com/en/user-guide/intro-key-concepts#connecting-to-snowflake

AI/ML & App Development Features

Feature	What It Does	Key Detail
Snowflake Notebooks	Interactive Python/SQL/Markdown in Snowsight	Run on WH or SPCS, Git integration
Streamlit in Snowflake	Build data apps in Python	Runs in Snowflake, no infra mgmt, uses warehouse
Snowpark	DataFrame API (Python, Scala, Java)	Push-down processing; runs IN Snowflake, not client
Cortex AI_COMPLETE	LLM text generation	SQL function, serverless
Cortex AI_CLASSIFY	Text classification	SQL function, scalable
Cortex AI_SENTIMENT	Sentiment score (-1 to 1)	SQL function
Cortex AI_SUMMARIZE	Summarize text	SQL function
Cortex AI_TRANSLATE	Translate between languages	SQL function
Cortex AI_EXTRACT	Extract entities from text	SQL function
Cortex AI_REDACT	Redact PII from text	SQL function
Cortex AI_PARSE_DOCUMENT	Extract text from PDFs/images	OCR mode or LAYOUT mode
Cortex AI_EMBED	Generate vector embeddings	SQL function
Cortex Search	Hybrid vector+keyword search	Creates CORTEX_SEARCH_SERVICE object
Cortex Analyst	Natural language to SQL	Requires semantic model YAML on stage
Cortex Agents	Multi-tool AI orchestrator	Combines Search + Analyst + tools
Snowflake ML	Model registry + feature store	Register, version, deploy ML models

- ML Functions (4 serverless, create MODEL OBJECTS): FORECAST, ANOMALY_DETECTION, CLASSIFICATION, TOP_INSIGHTS

https://docs.snowflake.com/en/user-guide/snowflake-cortex/aisql

Snowpark Container Services (SPCS)

- Run Docker containers directly inside Snowflake
- Components: Compute Pool (GPU/CPU), Service (container), Service Endpoint (ingress)
- Image Registry built into Snowflake; data never leaves Snowflake boundary

https://docs.snowflake.com/en/developer-guide/snowpark-container-services/overview

Native App Framework

- Package data + code + UI into installable app via Marketplace
- **Application Package** = container for app code, data, metadata
- **manifest.yml** = required file; **setup_script.sql** = SQL that runs on install
- Runs in consumer's account (consumer pays compute)

https://docs.snowflake.com/en/developer-guide/native-apps/native-apps-about

Snowflake Notebooks (NEW to COF-C03)

- Jupyter-style notebooks in Snowsight; SQL, Python, Markdown cells
- Run on warehouses or SPCS (container runtime); Git integration for version control

https://docs.snowflake.com/en/user-guide/ui-snowsight/notebooks

Git Integration (NEW to COF-C03)

- CREATE GIT REPOSITORY = stage-like, read-only object linked to external Git repo
- Can fetch files from Git directly into Snowflake; READ-ONLY (cannot push back)

https://docs.snowflake.com/en/developer-guide/git/git-setting-up

File Format Objects

- Named database object defining data file properties; reusable across COPY INTO and stages
- Supported types: CSV, JSON, AVRO, ORC, PARQUET, XML

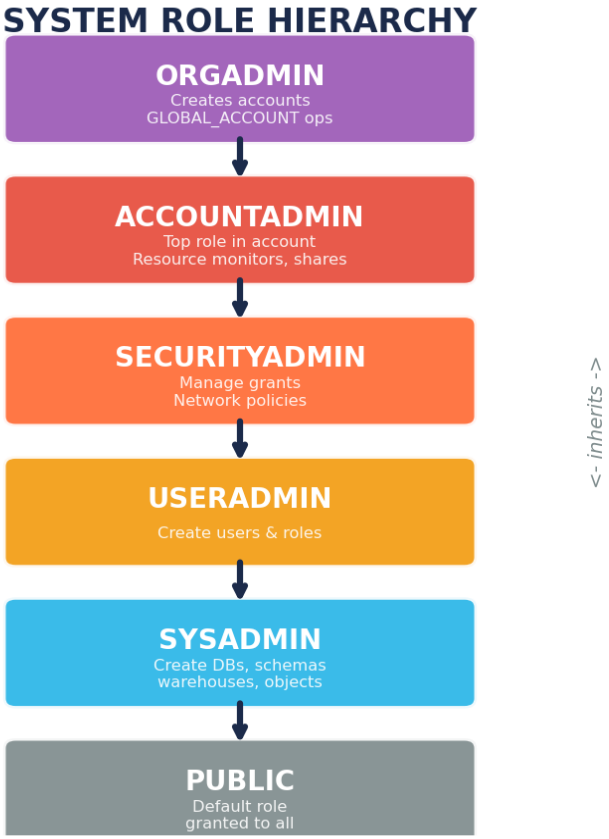
https://docs.snowflake.com/en/sql-reference/sql/create-file-format

Copilot (Being Replaced by Cortex Code)

- Natural language to SQL in Snowsight; requires COPILOT_USER database role in SNOWFLAKE db

https://docs.snowflake.com/en/user-guide/snowflake-copilot

2. ACCOUNT MANAGEMENT & DATA GOVERNANCE (20%)



ORGADMIN = GLOBAL_ACCOUNT anything! Not ACCOUNTADMIN!

System Role Responsibilities

Role	Key Responsibilities
ORGADMIN	Create accounts, org usage, SYSTEM$GLOBAL_ACCOUNT_SET_PARAMETER, enable replication
ACCOUNTADMIN	Top within account, resource monitors, shares, ACCOUNT_USAGE, billing, reader accounts
SECURITYADMIN	MANAGE GRANTS (global!), network policies, inherits USERADMIN
USERADMIN	CREATE USER + CREATE ROLE only (cannot MODIFY users -- need OWNERSHIP)
SYSADMIN	Create DBs, schemas, tables, warehouses, all data objects
PUBLIC	Default role, automatically granted to every user

https://docs.snowflake.com/en/user-guide/security-access-control-overview#system-defined-roles

Access Control: RBAC + DAC + UBAC

Model	How It Works
RBAC (Role-Based)	Privileges granted TO roles, roles granted TO users
DAC (Discretionary)	Object OWNER can grant access; owner = role active at CREATE time
UBAC (User-Based)	Privileges granted directly to users (active with USE SECONDARY ROLES ALL)
Combined	Snowflake uses ALL THREE

https://docs.snowflake.com/en/user-guide/security-access-control-overview

Critical Access Control Rules

- **Custom roles -> grant TO SYSADMIN** (not ACCOUNTADMIN) for object access chain
- **Objects owned by ROLE**, not user (role active at CREATE = owner)
- System role privileges CANNOT be revoked
- Managed access schema: owners lose grant ability; only schema owner + MANAGE GRANTS holders can grant
- **Future grants**: pre-grant privileges on objects not yet created in a schema/database
- No 'super-user' concept -- ALL access requires explicit grants

Role Types (6 Types)

Type	Scope	Detail
Account Roles	Account-wide	System-defined + custom; granted to users directly
Database Roles	Single database	Created IN a database; must be granted to account role to activate
Instance Roles	Class instances	For class instances, granted to account role
Application Roles	Native Apps	For Native App consumers
System Application Roles	Snowflake-provided	E.g., Budgets, DMFs
Service Roles	SPCS	For service endpoints

https://docs.snowflake.com/en/user-guide/security-access-control-overview#types-of-roles

Primary vs Secondary Roles

- **Primary Role** = current active role (for CREATE operations, ownership)
- **Secondary Roles** = additional roles activated for authorization (not for CREATE)
- USE SECONDARY ROLES ALL = activate all granted roles

Secondary roles let you access objects owned by different roles WITHOUT switching roles!

https://docs.snowflake.com/en/user-guide/security-access-control-overview#authorization-through-primary-role-and-secondary-roles

Managed Access Schemas

- Regular schemas: object OWNER can grant privileges on their objects
- Managed access: ONLY schema owner or MANAGE GRANTS role can grant; CREATE SCHEMA ... WITH MANAGED ACCESS

https://docs.snowflake.com/en/user-guide/security-access-control-configure#managed-access-schemas

Account Identifiers

Identifier	Format	Example
Account Name (preferred)	orgname-account_name	myorg-account1
Account Locator (legacy)	Short alphanumeric	xy12345
Connection URL	account_id.snowflakecomputing.com	myorg-account1.snowflakecomputing.com
Region ID	Cloud + region	aws_us_east_1, azure_westus2

https://docs.snowflake.com/en/user-guide/admin-account-identifier

Authentication Methods

Method	Details
Username/Password	Basic; JDBC default authenticator = "snowflake"
MFA	All editions; Duo push; ALLOW_CLIENT_MFA_CACHING = account level only
Key Pair	RSA 2048-bit; used by Kafka connector, SnowSQL, programmatic access
OAuth	External (Okta, Azure AD) or Snowflake OAuth via security integration
Federated/SSO	SAML 2.0; centralizes auth; separates authentication from access control
SCIM	User/group lifecycle provisioning (NOT authentication) -- all editions

https://docs.snowflake.com/en/user-guide/authentication

Network Policies & Network Rules

- **Network Policy** = IP allow/block lists; applied at Account, User, or Security Integration level
- **Network Rule** (newer) = more flexible, supports HOST_PORT, private endpoints
- SECURITYADMIN or ACCOUNTADMIN to create/apply; user-level overrides account-level
- At least 1 IP in allowed_ip_list or you lock yourself out!

https://docs.snowflake.com/en/user-guide/network-policies

Privileges Quick Reference

MANAGE GRANTS = GLOBAL (not database/schema level!) -- EXAM TRAP!

Category	Privilege	What It Does
Warehouse	USAGE	Run queries using warehouse
Warehouse	OPERATE	Suspend/resume + abort queries (NO resize)
Warehouse	MONITOR	View activity/metrics
Warehouse	MODIFY	Change properties including SIZE, auto-suspend, auto-resume
Warehouse (global)	MANAGE WAREHOUSES	MOM: Modify+Operate+Monitor on ALL WHs
Task	OPERATE / OWNERSHIP	Suspend/resume task
Task (global)	EXECUTE TASK	Run any task (account-level global priv)
Search Opt	ADD SEARCH OPTIMIZATION	On schema + OWNERSHIP on table
Sharing	CREATE SHARE / IMPORT SHARE	Create/import shares (ACCOUNTADMIN)
Global	MANAGE GRANTS	Grant any privilege to any role (GLOBAL only!)
Clone	COPY GRANTS	Copy all privileges EXCEPT OWNERSHIP on clone

https://docs.snowflake.com/en/user-guide/security-access-control-privileges

SHOW GRANTS Syntax

Command	Shows
SHOW GRANTS TO ROLE x	What privileges role x HAS
SHOW GRANTS OF ROLE x	WHO holds role x (users and parent roles)
SHOW GRANTS ON <object>	Privileges granted on a specific object
SHOW FUTURE GRANTS IN SCHEMA s	Pre-configured grants on future objects

https://docs.snowflake.com/en/sql-reference/sql/show-grants

Encryption & Key Management

- Auto-encrypts: **tables + internal stages** (NOT external stages by default)
- PUT encrypts **before leaving user's machine** (128-bit AES client-side, 256-bit server-side)
- Key rotation = ALL editions (automatic every 30 days)
- **Periodic rekeying** = Enterprise+ (re-encrypts data with new key annually)
- **Tri-Secret Secure** = Business Critical+ (customer key + Snowflake key = composite master key)
- All data encrypted at rest AND in transit (TLS 1.2+)

https://docs.snowflake.com/en/user-guide/security-encryption

All 8+ Policy Types

Policy	Purpose	Edition
Masking Policies	HIDES column values (tables, views, mat views -- NOT pipes)	Enterprise+
Row Access Policies	FILTERS rows by user/role	Enterprise+
Projection Policies	Prevents SELECT on specific columns	Enterprise+
Aggregation Policies	Forces queries to aggregate (no individual rows)	Enterprise+
Privacy Policies	Combines row access + aggregation in one	Enterprise+
Authentication Policies	Control how users authenticate (MFA, password, key pair)	All
Session Policies	Control session timeout, idle time	All
Password Policies	Password complexity, history, lockout rules	All
Packages Policies	Control which Python packages allowed in UDFs/SPs	All

https://docs.snowflake.com/en/user-guide/authentication-policies

Data Governance Features

Feature	Purpose	Edition
Data Classification	FINDS sensitive data (PII) automatically	Enterprise+
Object Tags	Key-value LABELS for governance/compliance/cost	All (some Enterprise+)
Tag-based Masking	Auto-apply masking policy via tag	Enterprise+
Trust Center	Security posture dashboard; scanner packages; CIS benchmarks	Enterprise+
Data Lineage	Tracks data flow (OBJECT_DEPENDENCIES view)	Enterprise+
Access History	Records column READ/WRITE access	Enterprise+
Alerts	Condition-based triggers (WHEN condition THEN action)	All
Notifications	Email/webhook/queue delivery for alerts and tasks	All

https://docs.snowflake.com/en/user-guide/governance

Logging, Tracing & Monitoring

Feature	Detail
Event Tables	Store log/trace data from UDFs, SPs, Streamlit; query with SQL
SYSTEM$SET_EVENT_TABLE()	Configure event table for an account
Trace Level	ALWAYS, ON_EVENT, OFF; controls granularity
Log Level	TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF
Query History	ACCOUNT_USAGE.QUERY_HISTORY or INFORMATION_SCHEMA.QUERY_HISTORY()
Login History	ACCOUNT_USAGE.LOGIN_HISTORY tracks authentication events

https://docs.snowflake.com/en/developer-guide/logging-tracing/logging-tracing-overview

Alerts & Notifications

- **Alerts**: condition-based triggers -- WHEN condition THEN action; evaluated on schedule (CRON/interval); require warehouse
- **Notifications**: email, webhooks, cloud queues (SNS, Event Grid, Pub/Sub); Notification Integration = named object

https://docs.snowflake.com/en/user-guide/alerts

ACCESS_HISTORY View

- In ACCOUNT_USAGE schema (Enterprise+); records which columns were READ and WRITTEN

- 3-hour latency, 1-year retention; requires GOVERNANCE_VIEWER database role

https://docs.snowflake.com/en/sql-reference/account-usage/access_history

Trust Center

- Security posture management in Snowsight; runs SCANNERS (CIS Benchmarks, Security Essentials, Threat Intelligence)
- Produces FINDINGS with severity levels; can schedule scanners and set up notifications

https://docs.snowflake.com/en/user-guide/trust-center

Resource Monitors & Cost

Action	Running Queries	Warehouse
Notify	Nothing -- alert only	Keeps running
Notify & Suspend	Queries COMPLETE	Then suspends
Notify & Suspend Immediately	Queries CANCELLED	Suspends NOW

- Only 1 at account level | ACCOUNTADMIN creates | Account + warehouse level only (NOT user level)
- Monitors credit usage only (NOT storage costs); cannot monitor serverless compute

https://docs.snowflake.com/en/user-guide/resource-monitors

Credit Consumption Sources

- Virtual warehouses (compute); Cloud services (when >10% of daily compute)
- Serverless: Snowpipe, auto-clustering, mat view refresh, search optimization, replication, serverless tasks, ML functions, QAS, Snowpipe Streaming
- Snowpark Container Services (compute pools); Cortex AI functions

https://docs.snowflake.com/en/user-guide/credits

WAREHOUSE_METERING_HISTORY -- 3 Access Methods

- **ACCOUNT_USAGE view**: SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY -- 3hr latency, 1yr
- **INFORMATION_SCHEMA function**: TABLE(INFORMATION_SCHEMA.WAREHOUSE_METERING_HISTORY(...)) -- real-time, 7-day
- **Snowsight UI**: Admin > Warehouses > Usage tab

https://docs.snowflake.com/en/sql-reference/account-usage/warehouse_metering_history

Data Replication & Failover

- **Database replication**: replicate DB to another account/region (read-only replica)
- **Replication groups**: bundle DBs + shares + roles for replication
- **Failover**: promote replica to primary (Business Critical+); **Failback**: switch back
- ORGADMIN enables replication between accounts; Client redirect: redirect connections during failover
- Supports: databases, shares, account params, network policies, users, roles, warehouses, resource monitors, integrations

https://docs.snowflake.com/en/user-guide/replication-intro

Storage & API Integrations

- **Storage Integration**: named account-level object linking Snowflake to S3/GCS/Azure; avoids passing credentials
- **API Integration**: connects to external REST APIs for external functions; defines allowed endpoints + auth

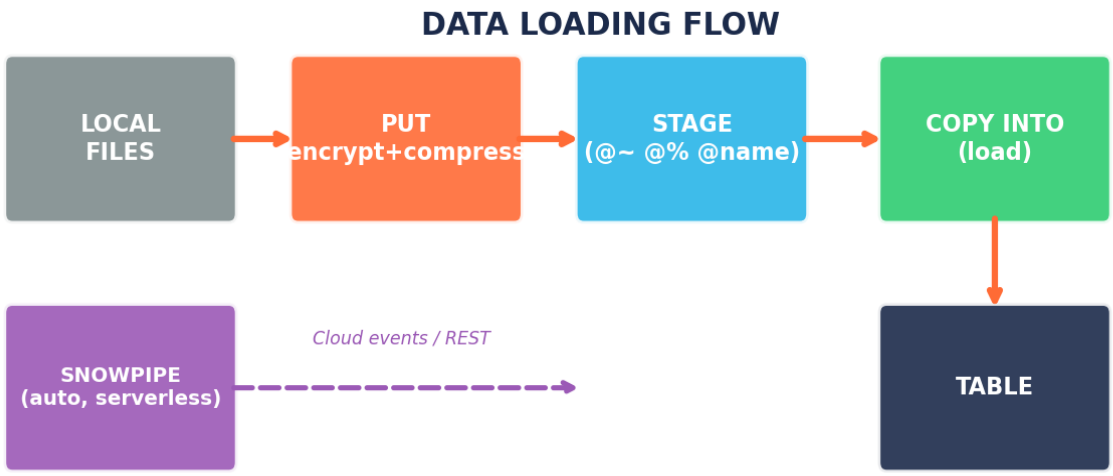
https://docs.snowflake.com/en/sql-reference/sql/create-storage-integration

Data Lineage & Query Attribution

- OBJECT_DEPENDENCIES view in ACCOUNT_USAGE -- tracks upstream/downstream dependencies (Enterprise+)
- QUERY_ATTRIBUTION_HISTORY view -- shows credit cost per query for cost optimization

https://docs.snowflake.com/en/user-guide/object-dependencies

3. DATA LOADING, UNLOADING & CONNECTIVITY (18%)



https://docs.snowflake.com/en/user-guide/data-load-overview

Stage Types

Stage	Syntax	Key Detail
User stage	@~	One per user, can't ALTER, can't set file format
Table stage	@%tablename	One per table, can't ALTER, can't set file format
Named internal	@stagename	Created by user, inside Snowflake storage, can set file format
Named external	@stagename	Points to S3/GCS/Azure Blob; requires storage integration

- **Directory tables**: auto-catalog of staged files; enable with DIRECTORY=(ENABLE=TRUE)
- Internal stages always encrypted; External: use TYPE=AWS_SSE_S3 or TYPE=AWS_SSE_KMS

Stage Drop: Internal = files GONE FOREVER | External = only definition dropped, files stay in cloud!

https://docs.snowflake.com/en/user-guide/data-load-local-file-system-create-stage

PUT Command -- Deep Details

Feature	Detail
What PUT does	Uploads files from LOCAL machine to an INTERNAL stage (not external!)
Syntax	PUT file://<path> @<stage>
Encryption	Encrypts BEFORE leaving client (128-bit AES client-side, 256-bit AES server-side)
Compression	AUTO_COMPRESS = TRUE by default (gzip)
Parallel upload	PARALLEL=4 default; up to 99 threads for large files
Overwrite	OVERWRITE=FALSE default; TRUE overwrites existing staged files
Cannot use with	External stages (S3/GCS/Azure) -- use cloud tools instead
SnowSQL only	PUT/GET only work in SnowSQL CLI or connectors (NOT in Snowsight SQL worksheet)
File splitting	Files <64MB uploaded in parallel; larger files auto-split

https://docs.snowflake.com/en/sql-reference/sql/put

Bulk Loading File Sizing

Scenario	Recommended Size	Why
Bulk COPY INTO	100-250 MB compressed	Optimal parallelism per file; too small = overhead; too large = slow retry
Snowpipe	100-250 MB compressed	Same as bulk; smaller files = higher Snowpipe overhead
Very large files	Split into 100-250 MB chunks	Each file loaded by one thread; splitting enables parallel loading
Too many small files	Aggregate before staging	Thousands of tiny files = massive overhead, slow performance

https://docs.snowflake.com/en/user-guide/data-load-considerations-prepare#file-sizing-best-practices

COPY INTO (Load) Rules

- Tracks history 64 days (skips loaded files) | **FORCE=TRUE** reloads all
- **CAN** transform: reorder, omit, cast, truncate columns, SUBSTR, CONCAT
- **CANNOT**: filter (WHERE), join, aggregate, sort, GROUP BY
- PURGE=TRUE deletes staged files after load | ON_ERROR: CONTINUE, SKIP_FILE, SKIP_FILE_n, SKIP_FILE_n%, ABORT_STATEMENT (default)
- SIZE_LIMIT: max bytes; MATCH_BY_COLUMN_NAME: matches by NAME not position

https://docs.snowflake.com/en/sql-reference/sql/copy-into-table

COPY INTO (Unload) Rules

COPY INTO unloads to STAGES ONLY (internal + external) -- NOT local dirs, NOT REST!

- **SINGLE = FALSE** (default) = multiple files | SINGLE=TRUE = one file (5GB max)
- PARTITION BY splits rows into files by expression (NOT table partitioning)
- GET command downloads files from internal stages to local filesystem

https://docs.snowflake.com/en/sql-reference/sql/copy-into-location

VALIDATE vs VALIDATION_MODE

	VALIDATE()	VALIDATION_MODE
When	AFTER COPY INTO (post-load)	DURING COPY INTO (preview)
Does	Returns errors from last loaded files	Validates without loading
Transform	N/A	CANNOT use with transformations
Modes	VALIDATE(table, job_id)	RETURN_n_ROWS, RETURN_ERRORS, RETURN_ALL_ERRORS

https://docs.snowflake.com/en/sql-reference/functions/validate

File Formats

Format	Type	Key Options
CSV/TSV	Structured, FASTEST load	FIELD_OPTIONALLY_ENCLOSED_BY, SKIP_HEADER, SKIP_BYTE_ORDER_MARK
JSON	Semi-structured	STRIP_OUTER_ARRAY, STRIP_NULL_VALUES; default compression = AUTO (gzip)
Parquet	Columnar	Snappy default compression; query-optimized; good for analytics
Avro	Semi-structured	Deflate compression; schema evolution support
ORC	Columnar	Hadoop ecosystem; Zlib/Snappy compression
XML	Semi-structured	Can load into VARIANT column

- CSV = FASTEST format for loading (not Parquet!); LZO = Parquet unload compression; Snappy = Parquet load

https://docs.snowflake.com/en/sql-reference/sql/create-file-format

Snowpipe (Continuous Loading)

- Object = **PIPE** (not stage, not stream) | Serverless (no warehouse needed)
- Triggers: cloud events (SQS/GCS Pub-Sub/Azure Event Grid) + REST API (insertFiles)
- Recommended file: 100-250 MB | CREATE OR REPLACE resets pipe = may duplicate!
- **AUTO_INGEST=TRUE** for cloud event triggers | SYSTEM$PIPE_STATUS() to check state
- ON_ERROR default = **SKIP_FILE** (different from bulk COPY default of ABORT_STATEMENT)
- Error notifications via cloud messaging (SNS, Event Grid, Pub/Sub); VALIDATE_PIPE_LOAD to check errors

https://docs.snowflake.com/en/user-guide/data-load-snowpipe-intro

Snowpipe vs Snowpipe Streaming Comparison

Feature	Snowpipe (Classic)	Snowpipe Streaming
Trigger	Cloud events (SQS/GCS/Azure) or REST insertFiles	SDK call (Java, Python) or REST API
Data path	Files staged -> COPY INTO	Rows written directly to table (no files)
Latency	~1 minute (file detection + load)	~5 seconds (near real-time)
Throughput	Good for batch files	Up to 10 GB/s
Delivery	At-least-once (dedup by filename)	Exactly-once (offset tokens + channels)
Compute	Serverless (no warehouse)	Serverless (no warehouse)
Object	PIPE object	No Snowflake object needed
Best for	Continuous file-based ingestion	Real-time streaming from apps/IoT

https://docs.snowflake.com/en/user-guide/data-load-snowpipe-streaming-overview

Streams -- Complete Details

Feature	Detail
What it is	CDC tracker on tables, views, directory tables, ext tables
Key columns	METADATA$ACTION, METADATA$ISUPDATE, METADATA$ROW_ID (NO ISREAD!)
Consume rule	ONLY by committed DML reading stream; SELECT does NOT advance offset
Stale after	DATA_RETENTION_TIME_IN_DAYS (default 14d) -- unusable once stale
Standard	Default type -- tracks INSERT, UPDATE, DELETE
Append-only	Tracks INSERT only
Insert-only	For external tables and Iceberg tables
Check data	SYSTEM$STREAM_HAS_DATA() = TRUE if unconsumed changes

https://docs.snowflake.com/en/user-guide/streams-intro

Dynamic Tables -- Complete Details

Feature	Detail
What it is	Declarative data pipeline: define target as SELECT, Snowflake keeps it fresh
TARGET_LAG	Max staleness: '10 minutes', '1 hour', or 'DOWNSTREAM'
Replaces	Streams+tasks pattern for many ETL pipelines; simpler, less code
Refresh mode	Incremental or full refresh (auto-determined by Snowflake)
Chaining	DT1 -> DT2 -> DT3 (DAG of transformations); each DT is a node
Warehouse	Requires warehouse for refresh; can specify target warehouse
Monitoring	DYNAMIC_TABLE_REFRESH_HISTORY() in INFORMATION_SCHEMA
Limitations	No stored procs, no non-deterministic funcs, no external funcs in definition
vs Mat Views	DT can have JOINs, subqueries, complex SQL; Mat Views = single table only
DML	No DML allowed on dynamic tables (read-only, Snowflake manages data)

https://docs.snowflake.com/en/user-guide/dynamic-tables-about

Directory Tables -- Complete Details

Feature	Detail
What it is	Auto-catalog of files on a stage; like a read-only table listing staged files
Enable	ALTER STAGE @stg SET DIRECTORY=(ENABLE=TRUE) or at creation
Query	SELECT * FROM DIRECTORY(@stage_name)
Columns	RELATIVE_PATH, SIZE, LAST_MODIFIED, FILE_URL, ETAG
Refresh	ALTER STAGE @stg REFRESH -- must manually refresh to see new files
Auto-refresh	Can configure auto-refresh with cloud event notifications
Works on	Internal stages AND external stages
Use case	Catalog unstructured files (images, PDFs, videos) on stages
Streams on	Can create streams on directory tables to track file changes
NOT a table	It is a property of a stage, not a standalone object

https://docs.snowflake.com/en/user-guide/data-load-dirtables

Connectors & Integrations

Type	Examples / Details
Snowflake Drivers	JDBC, ODBC, Node.js, Go, .NET -- connect apps to Snowflake
Snowflake Connectors	Python connector, Spark connector, Kafka connector (key pair auth)
Storage Integration	Named object linking Snowflake to S3/GCS/Azure; avoids passing credentials
API Integration	Connects to external REST APIs (for external functions)
Git Integration	Connect Git repo to Snowflake; CREATE GIT REPOSITORY; sync files; READ-ONLY
Notification Integration	Email, webhooks, cloud queues for alerts/pipe notifications

https://docs.snowflake.com/en/user-guide/ecosystem

4. PERFORMANCE OPTIMIZATION & QUERYING (21%)

Warehouse Sizing Decision

Problem	Solution	Key Detail
Queries slow (complex)	SCALE UP (bigger WH)	More CPU/memory per node
Queries queuing (many users)	SCALE OUT (multi-cluster)	Enterprise+ only; more clusters
Spilling to local/remote disk	SCALE UP	Needs more memory; check Query Profile
Point lookups slow	Search Optimization	Enterprise+, serverless, equality/IN/GEOGRAPHY
Large scans, selective filters	Query Acceleration	Enterprise+, serverless, offloads scan portions

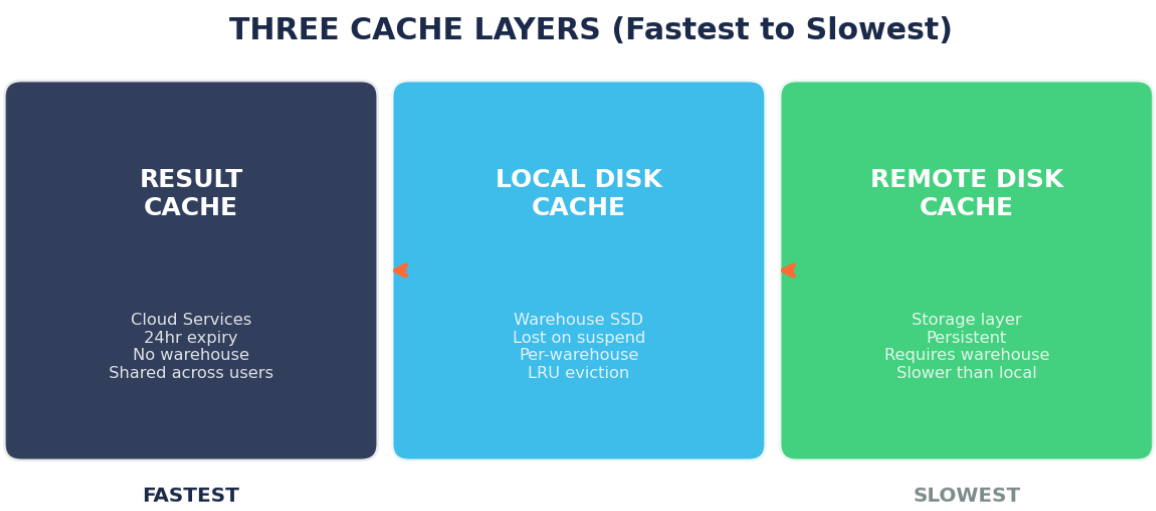
https://docs.snowflake.com/en/user-guide/warehouses-considerations

Multi-Cluster Warehouses (Enterprise+)

Mode	When	Scaling Policy Details
Auto-scale	Unknown/variable workload	STANDARD: add cluster immediately, shut down 2-3 checks idle
Maximized	Known high concurrency	ECONOMY: wait 6 min sustained load, shut down 5-6 checks idle

https://docs.snowflake.com/en/user-guide/warehouses-multicluster

Three Cache Layers



Result cache: invalidated by DATA CHANGE or 24HR -- NOT by warehouse/user change!

- **Metadata cache**: always on in Cloud Services; stores COUNT(*), MIN, MAX; no warehouse needed
- SELECT COUNT(*) FROM table = metadata only, zero cost if result cache not used

https://docs.snowflake.com/en/user-guide/querying-persisted-results

Optimization Services (Enterprise+)

Service	Best For	Key Detail
Search Optimization	Point lookups (=, IN, GEOGRAPHY, SUBSTRING, VARIANT)	Serverless; ADD SEARCH OPTIMIZATION ON table
Query Acceleration (QAS)	Large scans with selective filters	Serverless; SCALE_FACTOR (1-100) controls max compute

Service	Best For	Key Detail
Clustering Keys	Multi-TB tables, range queries	Medium cardinality; depth=1 optimal; AUTO reclustering
Materialized Views	Pre-compute expensive subqueries	Serverless refresh; single table; Enterprise+

https://docs.snowflake.com/en/user-guide/performance-optimization

Query Acceleration Service (QAS) -- Deep Details

- Offloads portions of query scanning to serverless compute; SCALE_FACTOR = 1 to 100
- SYSTEM$ESTIMATE_QUERY_ACCELERATION(<query_id>) = estimate benefit
- QUERY_ACCELERATION_ELIGIBLE ACCOUNT_USAGE view = find queries that would benefit
- Enable per warehouse: ALTER WAREHOUSE SET ENABLE_QUERY_ACCELERATION = TRUE

https://docs.snowflake.com/en/user-guide/query-acceleration-service

Search Optimization -- Sub-Properties

- EQUALITY -- for point lookups (WHERE col = value)
- SUBSTRING -- for LIKE, ILIKE, REGEXP patterns
- GEO -- for geospatial functions (ST_CONTAINS, etc.)
- VARIANT equality -- for semi-structured data lookups
- SYSTEM$ESTIMATE_SEARCH_OPTIMIZATION_COSTS(<table>) to estimate cost

https://docs.snowflake.com/en/user-guide/search-optimization-service

Query Profile Operators & Performance Tuning

Operator	What It Does
TableScan	Reads data from table
Filter	Applies WHERE conditions
Aggregate	GROUP BY / aggregation functions
JoinFilter	Bloom filter for join optimization
Sort / SortWithLimit	ORDER BY / TOP N with ORDER BY
Projection	SELECT column list
WindowFunction	OVER() window operations
ExternalScan	Reads external table data
Result	Final query output
WithClause/WithReference	CTE definition and usage

- Bytes spilled to LOCAL DISK -> Scale UP (needs more memory)
- Bytes spilled to REMOTE DISK -> Scale UP more (very expensive!)
- Partitions scanned >> total -> Add clustering key
- Exploding joins -> Fix JOIN condition; add filters

https://docs.snowflake.com/en/user-guide/ui-query-profile

Approximate Functions

- APPROX_COUNT_DISTINCT -- approximate COUNT(DISTINCT) using HyperLogLog (~2% error)
- APPROX_PERCENTILE -- approximate percentile
- APPROX_TOP_K -- approximate most frequent values; HLL = same as APPROX_COUNT_DISTINCT but returns state
- Use case: speed matters on billions of rows, exact count not needed

https://docs.snowflake.com/en/sql-reference/functions/approx_count_distinct

ACCOUNT_USAGE vs INFORMATION_SCHEMA

	ACCOUNT_USAGE (SNOWFLAKE DB)	INFORMATION_SCHEMA (per DB)
Scope	Account-wide (all DBs)	Single database only
Latency	45min - 3hr delay	Real-time (no delay)
Retention	365 days	7-14 days
Dropped objects	INCLUDED	EXCLUDED
Default role	ACCOUNTADMIN	Any role with access
Type	Views (can JOIN)	Table functions (most)

- 4 SNOWFLAKE database roles: OBJECT_VIEWER, USAGE_VIEWER, GOVERNANCE_VIEWER, SECURITY_VIEWER

https://docs.snowflake.com/en/sql-reference/account-usage#differences-between-account-usage-and-information-schema

Workload Management Best Practices

- Separate warehouses for different workloads (ETL vs BI vs ad-hoc)
- Group similar workloads to maximize cache reuse
- Use auto-suspend (5-10 min for frequent; 0 for constant workloads)
- Monitor with WAREHOUSE_METERING_HISTORY and QUERY_HISTORY views

https://docs.snowflake.com/en/user-guide/warehouses-considerations

4b. DATA TRANSFORMATIONS (included in Performance domain)

Semi-Structured Data

- Stored as **VARIANT** (also OBJECT, ARRAY types) | 128 MB max uncompressed per value
- Syntax: column:Element.SubElement (colon first, then dots for nesting)
- Column names: case-insensitive | JSON Keys: **CASE-SENSITIVE**
- **Arrays are 0-indexed**: first=[0], second=[1], third=[2]
- PARSE_JSON(NULL)=SQL NULL | PARSE_JSON('null')=VARIANT null (different!)
- FLATTEN output columns: **S**EQ, **K**EY, **P**ATH, **I**NDEX, **V**ALUE, **T**HIS
- :: for casting: column:key::STRING, column:key::NUMBER
- LATERAL FLATTEN = most common pattern for exploding nested arrays
- Snowflake auto-extracts up to 200 elements per partition into columnar form

https://docs.snowflake.com/en/user-guide/semistructured-intro

Semi-Structured Functions Reference

Function	Input -> Output	Use Case
OBJECT_CONSTRUCT()	Separate values -> VARIANT object	Build JSON from columns: OBJECT_CONSTRUCT('name', col1, 'age', col2)
TO_JSON()	VARIANT -> STRING	Convert VARIANT to JSON string for export/display
PARSE_JSON()	STRING -> VARIANT	Convert JSON string into queryable VARIANT
TO_VARIANT()	Any type -> VARIANT	Wrap scalar value as VARIANT
ARRAY_CONSTRUCT()	Values -> ARRAY	Build array: ARRAY_CONSTRUCT(1,2,3)
ARRAY_AGG()	Column values -> ARRAY	Aggregate rows into single array
FLATTEN()	VARIANT/ARRAY -> rows	Explode nested structure into rows
LATERAL FLATTEN	Nested array -> joined rows	Most common: SELECT * FROM t, LATERAL FLATTEN(t.arr)

Type Casting Functions (TO_ and TRY_)

Function	Does	TRY_ Version
TO_VARCHAR() / TO_CHAR()	Any -> STRING	TRY_TO_VARCHAR() -- returns NULL on failure
TO_NUMBER() / TO_DECIMAL()	String -> NUMBER	TRY_TO_NUMBER() -- returns NULL on failure
TO_DATE()	String/timestamp -> DATE	TRY_TO_DATE() -- returns NULL on failure
TO_TIMESTAMP()	String/number -> TIMESTAMP	TRY_TO_TIMESTAMP() -- returns NULL on failure
TO_BOOLEAN()	String -> BOOLEAN	TRY_TO_BOOLEAN() -- returns NULL on failure
TO_BINARY()	String -> BINARY	TRY_TO_BINARY() -- returns NULL on failure
TO_TIME()	String -> TIME	TRY_TO_TIME() -- returns NULL on failure

- **TO_** functions throw ERROR on invalid input | **TRY_** functions return NULL instead
- OBJECT_CONSTRUCT builds VARIANT from values; TO_JSON converts VARIANT to string (two-step: build then stringify)

Unstructured Data

- Stored on stages (internal or external); NOT loaded into tables
- Access via **scoped URL** (temp, for current role), **file URL** (persistent), or **pre-signed URL** (no auth needed, expires)
- Directory tables catalog unstructured files on stages
- Process with UDFs, external functions, or Cortex AI (AI_PARSE_DOCUMENT)

https://docs.snowflake.com/en/user-guide/unstructured-intro

Snowpark DataFrames -- Complete Details

- Available in Python, Java, Scala
- **Lazy evaluation**: operations build a query plan, NOT executed immediately
- Execution triggered by: .collect(), .show(), .count(), .to_pandas(), writing to table
- **Pushdown**: all computation runs on Snowflake server (not client)
- Entry point: Session object; DataFrames are IMMUTABLE -- each transformation returns new DataFrame
- Can create UDFs and stored procedures from Snowpark code

https://docs.snowflake.com/en/developer-guide/snowpark/python/working-with-dataframes

Stored Procedures vs UDFs

	Stored Procedure	UDF	UDTF
Call	CALL sp_name()	In SELECT, WHERE, etc.	In FROM clause
Per statement	ONE per SQL	Multiple per SQL	Multiple per SQL
Returns	Single value	Scalar or tabular	Tabular (rows)
Languages	Java, JS, Python, Scala, SQL	Java, JS, Python, SQL	Java, JS, Python, SQL
Owner rights	Default = owner rights	Caller rights	Caller rights
DDL/DML	YES (can INSERT, CREATE)	NO (read-only)	NO

External functions = scalar UDF subtype, calls API OUTSIDE Snowflake via API integration

https://docs.snowflake.com/en/developer-guide/stored-procedure/stored-procedures-overview

Tasks

Feature	Detail
Schedule	CRON or INTERVAL (USING CRON / SCHEDULE = 10 MINUTE)
SQL per task	1 SQL only (use SP for multiple statements)
Task graph (DAG)	Max 100 predecessors + 100 successors; 1000 in graph
Created state	SUSPENDED by default -- must ALTER TASK ... RESUME
Overlap behavior	If prev run still going, next = SKIPPED (not queued)
Compute	User warehouse OR serverless (max 2XL for serverless)
Privileges	OPERATE/OWNERSHIP to suspend/resume
Execute manually	EXECUTE TASK taskname (runs immediately)
WHEN clause	WHEN SYSTEM$STREAM_HAS_DATA() -- only run if data
Check history	TASK_HISTORY() table function in INFORMATION_SCHEMA

https://docs.snowflake.com/en/user-guide/tasks-intro

SQL Transformations & Functions

Category	Examples / Key Points
Aggregate Functions	COUNT, SUM, AVG, MIN, MAX, LISTAGG, ARRAY_AGG, MEDIAN, APPROX_COUNT_DISTINCT
Window Functions	ROW_NUMBER(), RANK(), DENSE_RANK(), LEAD(), LAG(), NTILE(), FIRST_VALUE(), LAST_VALUE()
Window syntax	func() OVER (PARTITION BY col ORDER BY col ROWS BETWEEN...)
QUALIFY	Filter window function results (like HAVING for GROUP BY): QUALIFY ROW_NUMBER()... = 1
GENERATOR	TABLE(GENERATOR(ROWCOUNT => 100)) generates rows
LATERAL	Reference columns from preceding table in FROM clause
PIVOT / UNPIVOT	Rotate rows to columns / columns to rows
MERGE	Upsert: INSERT when not matched, UPDATE when matched
INSERT OVERWRITE	Truncate + insert atomically

https://docs.snowflake.com/en/sql-reference/constructs/qualify

Set Operators & Sampling

Operator / Feature	Returns
INTERSECT	Values in BOTH result sets (distinct)
UNION	All unique values (removes duplicates)
UNION ALL	All values INCLUDING duplicates (faster, no dedup)
MINUS / EXCEPT	Values in first but NOT in second
SAMPLE (10 ROWS)	Exactly 10 random rows (row-level sampling)
SAMPLE (10)	~10% of rows (block-level sampling, faster, approximate)
TABLESAMPLE BERNOULLI	Row-level probabilistic sampling
TABLESAMPLE SYSTEM / BLOCK	Block-level sampling (faster, less random)

https://docs.snowflake.com/en/sql-reference/constructs/sample

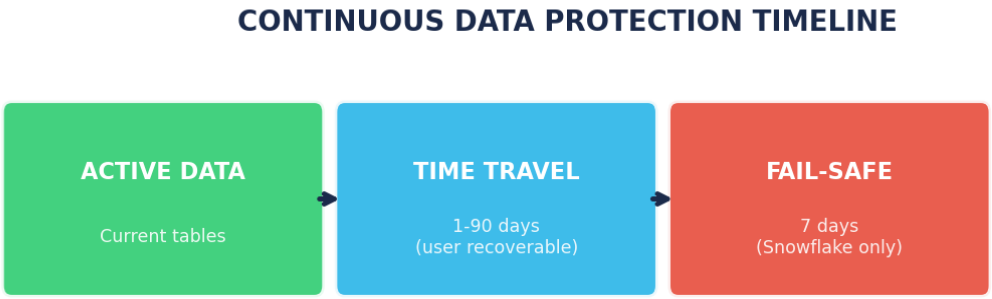
Views, Sequences, Loops

- ALTER VIEW **cannot change columns** -- must recreate (CREATE OR REPLACE)

- Secure views: recommended for sharing | Standard views CAN be shared but expose definition
- **Sequences**: auto-increment; CREATE SEQUENCE; nextval; may have gaps; NOT guaranteed contiguous
- LOOP requires BREAK | WHILE has condition | FOR has iterator | REPEAT until TRUE

https://docs.snowflake.com/en/sql-reference/sql/create-sequence

5. DATA COLLABORATION & PROTECTION (10%)



Time Travel & Fail-safe

	Time Travel	Fail-safe
Duration	1-90 days (Enterprise+ for >1 day)	7 days (permanent tables only, not configurable)
Who recovers	User (UNDROP, AT/BEFORE)	Snowflake support only
Query by	Statement ID, offset, timestamp (NOT session ID)	N/A
UNDROP needs	CREATE privilege + OWNERSHIP	Contact Snowflake
Transient/Temp	0-1 day Time Travel	NO fail-safe (0 days)
Iceberg tables	Yes	NO fail-safe

- AT(TIMESTAMP => ...) | AT(OFFSET => -300) | BEFORE(STATEMENT => '<query_id>') | UNDROP TABLE t

https://docs.snowflake.com/en/user-guide/data-time-travel

Zero-Copy Cloning

- Clones metadata only -- zero extra storage until data changes
- Can clone: databases, schemas, tables (permanent/transient), streams, tasks, sequences
- Privileges NOT cloned by default (use COPY GRANTS) | COPY GRANTS = all **except OWNERSHIP**
- Internal named stages NOT cloned (use INCLUDE INTERNAL STAGES)
- Fully qualified refs in views/SPs still point to SOURCE (not clone)
- Temporary -> Permanent: NOT allowed | Transient -> Permanent: NOT allowed
- Clone of clone = independent; Cannot clone: external tables, internal stages (without INCLUDE)

https://docs.snowflake.com/en/user-guide/tables-storage-considerations#label-cloning-tables

Data Sharing -- Complete Model

NO data copied or transferred -- consumer queries run on PRODUCER's data (zero-copy)

- Shareable: databases, tables, dynamic tables, external tables, Iceberg tables, views (regular/secure/materialized), UDFs (NOT stored procedures)
- **REFERENCE_USAGE**: grant on each referenced DB when view spans multiple DBs
- Same region = FREE | Cross-region/cloud = replication + egress fees

Consumer Type	Who	Who Pays Compute
Full Snowflake account	Existing Snowflake customer	CONSUMER pays own compute
Reader account	Non-Snowflake customer	PRODUCER pays all costs

- Reader accounts: can query + unload; CANNOT create shares, create DBs, or run tasks. SHOW MANAGED ACCOUNTS to list.

https://docs.snowflake.com/en/user-guide/data-sharing-intro

Direct Shares vs Listings vs Marketplace

Method	Detail
Direct Share	Account-to-account; CREATE SHARE; add objects; grant to specific accounts
Private Listing	Share with selected consumers via Marketplace UI; cross-region possible
Public Listing	Available to ALL Snowflake users on Marketplace; discoverable by anyone
Data Exchange	Curated group of accounts sharing data (deprecated in favor of listings)

- **Auto-Fulfillment**: allows cross-region/cloud sharing via automatic replication. Provider pays replication costs.

https://docs.snowflake.com/en/user-guide/data-sharing-provider

Data Clean Rooms

- Secure environment where 2+ parties analyze combined data WITHOUT sharing raw data
- Uses secure views, row access policies, and Snowflake-managed governance
- Common use: ad measurement, audience overlap, healthcare collaboration

https://docs.snowflake.com/en/collaboration/data-clean-rooms